

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

C++ MINI PROJECT

TIC TAC TOE GAME

A Beginner-Level Console Application Using C++

Submitted by: Sagar Gupta

UID: 25BCD10169

Programme: BCA – Data Science

University: Chandigarh University

Subject: C++ / Object-Oriented Programming

Submitted to: MR Abhinay Sood

Academic Session: 2026–2027

INDEX

Tic Tac Toe Game in C++

S. No.	Topic	Page
1	Introduction	3
2	Background and Overview of the Game	4
3	What is C++?	5
4	What is Object-Oriented Programming (OOP)?	6
5	About the Project	7
6	Problem Statement and Need	8
7	Objectives of the Project	9
8	Scope and Features	10
9	Hardware and Software Requirements	11
10	C++ Concepts Used	12
11	Game Rules and Board Representation	13
12	System Design and Program Logic	14
13	Algorithm and Flow of Execution	15
14	Working of the Project	16
15	Functions and Their Responsibilities	17
16	Source Code – Part 1	18
17	Source Code – Part 2	19
18	Sample Output and Test Cases	20
19	Advantages, Limitations and Future Enhancements	21
20	Conclusion and Bibliography	22

Note: Page numbers are approximate and may shift slightly if the document is edited or opened with different page settings.

1. INTRODUCTION

Tic Tac Toe is a simple and popular two-player strategy game played on a square grid containing nine spaces. The players take turns placing their symbols, usually X and O, in empty spaces. The first player to create a straight line of three matching symbols wins the game. A line may be formed horizontally, vertically, or diagonally. If all nine spaces are filled and neither player has formed a line, the game ends in a draw.

This mini project implements the familiar game as a console-based application using the C++ programming language. The program displays a 3 × 3 board, accepts a position from the current player, checks whether the selected cell is available, updates the board, and checks for a win or draw.

The project is intentionally designed for beginners. It avoids advanced libraries, graphics, file handling, and complex class structures. Instead, it demonstrates fundamental programming skills such as variables, arrays, functions, loops, conditional statements, and user input.

A small game is a useful learning exercise because the programmer can understand the complete flow from input to output. It also provides clear conditions that can be tested, such as invalid moves, occupied cells, winning combinations, and a draw.

The purpose of this report is to explain the idea, design, implementation, and testing of the Tic Tac Toe program in a structured way. The source code is included in two parts for readability.

2. BACKGROUND AND OVERVIEW OF THE GAME

Tic Tac Toe is also known by names such as Noughts and Crosses. It is commonly played with paper and pencil, but its rules can be implemented easily in a computer program. The game has a small state space: there are nine cells, and each cell can be empty, contain X, or contain O.

A computer version follows the same rules as the paper version. The main difference is that the computer program manages the board and validates each move. The players do not need to draw the grid themselves; they simply enter a number from 1 to 9 to select a cell.

The board positions in this project are numbered from left to right and top to bottom. Position 1 is the top-left cell, position 5 is the centre cell, and position 9 is the bottom-right cell. This numbering makes the game easy to operate using a keyboard.

Position	**Position in board**
1, 2, 3	Top row
4, 5, 6	Middle row
7, 8, 9	Bottom row

The game alternates between Player X and Player O. Player X takes the first turn. After a valid move, the program checks for a winner. If no player has won and empty cells remain, the turn changes. The game ends when a player wins or the board becomes full.

3. WHAT IS C++?

C++ is a general-purpose programming language used to develop a wide range of software, including desktop applications, games, operating systems, embedded systems, and performance-intensive programs. It was developed by Bjarne Stroustrup as an extension of the C programming language.

C++ supports procedural programming as well as object-oriented programming. It also provides features such as classes, objects, inheritance, polymorphism, templates, references, and standard library containers. These features allow programmers to write both small programs and large software systems.

A C++ program is generally written in a source file and translated by a compiler into an executable program. The compiler checks the source code for syntax and type errors. Once compilation is successful, the program can be run.

The Tic Tac Toe project uses a small set of C++ features. The iostream library provides input and output operations. Variables store the current player and number of moves. A two-dimensional character array stores the board. Functions separate the program into manageable tasks.

The statement `using namespace std;` allows standard library names such as `cout` and `cin` to be used without writing the `std::` prefix each time. The `main()` function is the starting point of the program.

C++ element	**Use in this project**
#include <iostream>	Enables keyboard input and console output
int main()	Starting point of program execution
cout	Displays messages and the board
cin	Reads the player's selected position
if / else	Checks conditions and changes turns
while	Repeats the game until it ends

4. WHAT IS OBJECT-ORIENTED PROGRAMMING (OOP)?

Object-Oriented Programming, commonly called OOP, is a programming approach in which a program is organised around objects. An object combines data and the functions that operate on that data. A class is a blueprint used to create objects.

OOP is commonly explained through four major principles: encapsulation, abstraction, inheritance, and polymorphism.

Principle	**Meaning**
Encapsulation	Combines data and related functions in a class and controls access to them.
Abstraction	Shows essential operations while hiding unnecessary implementation details.
Inheritance	Allows a new class to reuse and extend features of an existing class.
Polymorphism	Allows the same interface or function name to behave in different ways.

The current Tic Tac Toe program is written in a procedural style to keep it beginner-friendly. It does not define custom classes or demonstrate inheritance and polymorphism. However, C++ supports OOP, and the same project could later be redesigned using a Game class that stores the board, current player, and game status.

For example, a future class could contain methods such as displayBoard(), makeMove(), checkWin(), and startGame(). This would group the game's data and behaviour together. Learning the procedural version first makes it easier to understand the responsibilities that could later be placed inside a class.

5. WHAT IS THE PROJECT ABOUT?

The project is a two-player Tic Tac Toe game that runs in a command-line or terminal window. It allows two people to play on the same computer by taking turns. One player uses X and the other uses O.

At the beginning, the program creates a 3 × 3 board containing the digits 1 to 9. These digits act as position labels. The program displays the board and asks the current player to enter a position. If the position is valid and empty, the selected cell is replaced with the player's symbol.

After each move, the program checks all possible winning lines. There are three horizontal lines, three vertical lines, and two diagonal lines. If any line contains three matching symbols, the current player wins.

If a selected position is outside the range 1 to 9, the program displays an error message. If the selected cell already contains X or O, the program rejects the move. In both cases, the same player is asked to try again.

If nine valid moves are completed without a winner, the program declares a draw. The program then ends. This version does not include a computer opponent, a graphical interface, or a permanent scoreboard.

6. PROBLEM STATEMENT AND NEED

The aim of this project is to create a simple digital version of Tic Tac Toe that follows the standard rules and can be operated through keyboard input. The program must display the board clearly, accept player moves, prevent invalid selections, identify a winner, and detect a draw.

When a beginner writes a game program, several small problems must be handled correctly. The program must map a number entered by the user to the correct row and column of the board. It must also ensure that an already occupied cell is not overwritten.

Another important requirement is checking the winning condition. A player can win in eight possible ways: three rows, three columns, and two diagonals. The program must examine all these combinations after every valid move.

The need for this project is educational. It provides a practical example of how basic programming concepts work together to solve a problem. It also encourages logical thinking, step-by-step planning, testing, and debugging.

The project is suitable for an introductory programming assignment because the rules are familiar, the input is simple, and the output can be checked visually.

7. OBJECTIVES OF THE PROJECT

The main objective is to design and implement a working two-player Tic Tac Toe game using basic C++ programming concepts.

- To create and display a 3 × 3 game board using a two-dimensional array.
- To allow two players to play by entering positions from 1 to 9.
- To alternate turns between Player X and Player O after every valid move.
- To validate the player's input and reject positions outside the allowed range.
- To prevent a player from selecting a cell that is already occupied.
- To check horizontal, vertical, and diagonal winning combinations.
- To identify and display the winner when a valid winning combination is formed.
- To declare a draw when all nine cells are filled without a winner.
- To practise the use of arrays, functions, loops, conditions, and input/output statements.
- To develop confidence in writing, compiling, running, and testing a C++ program.

A successful result is a program that follows the rules consistently, responds clearly to user input, and terminates when a player wins or the board is full.

8. SCOPE AND FEATURES

The scope of this version is limited to a local, two-player game in a terminal. Both players use the same keyboard and take turns on the same board. The program does not require an internet connection or an external database.

Feature	Description
Two-player mode	Two people play on one computer.
3 × 3 board	The game contains nine selectable cells.
Player symbols	Player X and Player O use different symbols.
Turn switching	The program changes the active player after a valid move.
Input validation	Numbers outside 1–9 are rejected.
Occupied-cell check	A cell containing X or O cannot be selected again.
Win detection	Rows, columns, and diagonals are checked.
Draw detection	The game declares a draw after nine non-winning moves.

The project can be extended in the future. Possible additions include a restart option, player names, a score counter, a single-player mode, difficulty levels, and a graphical user interface. These features are not included in the present beginner version.

9. HARDWARE AND SOFTWARE REQUIREMENTS

The program has modest system requirements because it uses only standard console input and output. It does not need a dedicated graphics card, large memory capacity, or an internet connection.

Requirement	**Minimum / Suggested**
Processor	Any modern desktop or laptop processor
RAM	2 GB or more
Storage	A few megabytes for source code and executable
Operating system	Windows, Linux, or macOS
Compiler	GCC, MinGW, Clang, or another C++ compiler
IDE / editor	Visual Studio Code, Code::Blocks, Dev-C++, or Visual Studio
Language standard	C++11 or later
Input device	Keyboard
Output device	Monitor or terminal window

To compile the program with GCC, save the source code in a file such as `tictactoe.cpp`, open a terminal in that folder, and run:

```
g++ tictactoe.cpp -o tictactoe
```

On systems where the executable is named `tictactoe`, it can be started with `./tictactoe` on Linux or macOS. On Windows, run `tictactoe.exe` from the terminal or open it through the development environment.

10. C++ CONCEPTS USED

Concept	Explanation	Project use
Header file	A library declaration included in the program.	iostream is used for cin and cout.
Variable	A named memory location that stores a value.	choice, moves, and player store game information.
2D array	An array arranged in rows and columns.	board[3][3] stores nine cells.
Function	A named block of code that performs a task.	displayBoard() and checkWin() divide responsibilities.
Loop	Repeats a block of statements.	while repeats turns; for checks winning lines.
Condition	Runs code based on a true or false expression.	if checks input, cells, and winning combinations.
Boolean	A data type with true or false values.	checkWin() returns true or false.
Input/output	Reads input and displays information.	cin reads a move and cout displays messages.

A two-dimensional array is especially suitable for the board because the game itself is arranged in rows and columns. The first index identifies the row and the second index identifies the column.

Functions improve readability. Instead of writing the board display and winning checks repeatedly inside main(), the program calls separate functions for these tasks. This makes the main game loop easier to understand.

11. GAME RULES AND BOARD REPRESENTATION

The board contains three rows and three columns. At the start, every cell contains a number from 1 to 9. These numbers show the player which position to enter.

```
 1 | 2 | 3
---|---|---
 4 | 5 | 6
---|---|---
 7 | 8 | 9
```

The rules followed by the program are:

1. Player X always takes the first turn.
2. Players must choose a number between 1 and 9.
3. A player can select only an empty cell.
4. After a valid move, the program checks whether that player has won.
5. A player wins by placing three matching symbols in one row, one column, or one diagonal.
6. If the game has not been won, the next player gets a turn.
7. If the board becomes full and there is no winner, the result is a draw.

The program converts a selected position into array indexes using two expressions: row = (choice - 1) / 3 and col = (choice - 1) % 3. Integer division identifies the row, while the remainder identifies the column.

12. SYSTEM DESIGN AND PROGRAM LOGIC

The program is organised into three main parts: board storage, board display, and game control. The board is stored in a global two-dimensional character array. The displayBoard() function prints the current state of the board. The checkWin() function examines the possible winning lines.

The main() function controls the sequence of play. It initialises the current player as X and the move counter as zero. A while loop continues as long as fewer than nine valid moves have been made.

During each iteration, the program displays the board and asks the current player for a position. It validates the number and checks whether the selected cell is empty. If the move is invalid, the continue statement starts the next loop iteration without increasing the move counter or changing the player.

For a valid move, the program updates the selected cell and increases the move counter. It then calls checkWin(). If the function returns true, the board is displayed and the winner is announced. Otherwise, the player symbol changes from X to O or from O to X.

When the loop ends without a winner, all nine cells have been used. The program displays the final board and prints the draw message.

13. ALGORITHM AND FLOW OF EXECUTION

Algorithm:

8. Start the program.
9. Create a 3 × 3 character array containing the position labels 1 to 9.
10. Set the move counter to zero and the current player to X.
11. Display the board.
12. Ask the current player to enter a position from 1 to 9.
13. If the input is outside the range, show an error and ask again.
14. Convert the position into a row and column.
15. If the cell is already occupied, show an error and ask again.
16. Place the current player's symbol in the selected cell.
17. Increase the move counter.
18. Check all rows, columns, and diagonals for three matching symbols.
19. If a winning line exists, display the board and announce the winner, then stop.
20. If no winner exists, switch the current player.
21. Repeat until nine valid moves have been made.
22. If the board is full and no winner exists, display a draw message.
23. Stop the program.

Textual flow of execution:

```
START
  |
INITIALISE BOARD, PLAYER AND MOVE COUNT
  |
DISPLAY BOARD AND GET PLAYER INPUT
  |
IS INPUT VALID AND CELL EMPTY?
  | No --------------------> SHOW ERROR / ASK AGAIN
  | Yes
PLACE SYMBOL AND INCREASE MOVE COUNT
  |
CHECK WINNING COMBINATIONS
  | Win -------------------> DISPLAY WINNER -> END
  | No
ARE NINE MOVES COMPLETE?
  | No --------------------> SWITCH PLAYER -> REPEAT
  | Yes
DISPLAY DRAW
  |
END
```

14. WORKING OF THE PROJECT

When the program starts, it prints the title of the game and displays the empty board with numbered positions. The first prompt asks Player X to enter a position. Suppose Player X enters 1. The program calculates row 0 and column 0, then replaces the number 1 with X.

After the move, the program checks whether a winning line has been created. Since only one cell is occupied, no winning line exists. The active player changes to O. The board is displayed again, and Player O is asked to enter a position.

The same process continues for each valid move. The board always shows the latest state. If a player enters a position that has already been used, the program does not overwrite the existing symbol. It displays an error message and asks the same player to choose another position.

After every valid move, the program checks the eight possible winning lines. If the current player has formed a line of three symbols, the game immediately ends and displays the winner. If no winning line exists after the ninth valid move, the game ends with a draw.

This approach ensures that the game does not continue after a result has been decided. It also ensures that invalid attempts do not count as moves.

15. FUNCTIONS AND THEIR RESPONSIBILITIES

Function / component	Responsibility
displayBoard()	Prints all nine board cells in a 3 × 3 layout with separators.
checkWin()	Checks three rows, three columns, and two diagonals.
main()	Initialises the game, accepts input, validates moves, switches players, and ends the game.
board[3][3]	Stores the current symbols and unused position labels.
player	Stores whose turn it is: X or O.
moves	Counts the number of valid moves completed.
choice	Stores the position entered by the current player.
row and col	Store the calculated array indexes for the selected position.

The displayBoard() function does not change the game data. It only prints the contents of the array. The checkWin() function returns a Boolean result. It returns true if any winning combination is found and false otherwise.

The main() function contains the central control logic. It calls the other functions when required. Keeping these tasks separate makes the source code easier to read, understand, and maintain.

16. SOURCE CODE – PART 1

Header, board initialisation, display and win checking

```
#include <iostream>
using namespace std;

char board[3][3] = {
    {'1', '2', '3'},
    {'4', '5', '6'},
    {'7', '8', '9'}
};

void displayBoard() {
    cout << "\n";
    cout << " " << board[0][0] << " | " << board[0][1] << " | " << board[0][2] << endl;
    cout << "---|---|---" << endl;
    cout << " " << board[1][0] << " | " << board[1][1] << " | " << board[1][2] << endl;
    cout << "---|---|---" << endl;
    cout << " " << board[2][0] << " | " << board[2][1] << " | " << board[2][2] << endl;
    cout << "\n";
}

bool checkWin() {
    for (int i = 0; i < 3; i++) {
        if (board[i][0] == board[i][1] &&
            board[i][1] == board[i][2])
            return true;

        if (board[0][i] == board[1][i] &&
            board[1][i] == board[2][i])
            return true;
    }

    if (board[0][0] == board[1][1] &&
        board[1][1] == board[2][2])
        return true;

    if (board[0][2] == board[1][1] &&
        board[1][1] == board[2][0])
        return true;

    return false;
}
```

The first part includes the required input/output library and declares the board. The board is a character array because each cell stores one character: a digit, X, or O. The display function prints the board in a readable grid.

The win-checking function uses a loop to examine each row and column. Two additional conditions examine the diagonals. If none of the eight lines contains three equal characters, the function returns false.

17. SOURCE CODE – PART 2

Main function and game control

```
int main() {
    int choice;
    int moves = 0;
    char player = 'X';

    cout << "=======================\n";
    cout << "      TIC TAC TOE\n";
    cout << "=======================\n";

    while (moves < 9) {
        displayBoard();

        cout << "Player " << player << ", enter position (1-9): ";
        cin >> choice;

        if (choice < 1 || choice > 9) {
            cout << "Invalid position! Try again.\n";
            continue;
        }

        int row = (choice - 1) / 3;
        int col = (choice - 1) % 3;

        if (board[row][col] == 'X' || board[row][col] == 'O') {
            cout << "Position already occupied! Try again.\n";
            continue;
        }

        board[row][col] = player;
        moves++;

        if (checkWin()) {
            displayBoard();
            cout << "Congratulations! Player " << player << " wins!\n";
            return 0;
        }

        if (player == 'X')
            player = 'O';
        else
            player = 'X';
    }

    displayBoard();
    cout << "Game Draw!\n";

    return 0;
}
```

The main function controls the complete game. It starts with Player X and repeats turns while the move count is below nine. Invalid input and occupied positions are rejected with continue, so the same player retains the turn.

For a valid move, the program updates the array and checks for a winner. If there is no winner, the player changes. If all nine moves are used, the final board is displayed and the game is declared a draw.

18. SAMPLE OUTPUT AND TEST CASES

Example of a winning game:

```
=======================
      TIC TAC TOE
=======================

 1 | 2 | 3
---|---|---
 4 | 5 | 6
---|---|---
 7 | 8 | 9

Player X, enter position (1-9): 1
Player O, enter position (1-9): 4
Player X, enter position (1-9): 2
Player O, enter position (1-9): 5
Player X, enter position (1-9): 3

 X | X | X
---|---|---
 O | O | 6
---|---|---
 7 | 8 | 9

Congratulations! Player X wins!
```

Test	Input / situation	Expected result
1	Player enters 1	X or O is placed in top-left cell.
2	Player enters 0	Invalid position message; same turn continues.
3	Player enters 10	Invalid position message; same turn continues.
4	Player selects an occupied cell	Move is rejected; existing symbol remains.
5	A player completes a row	That player is announced as winner.
6	A player completes a column	That player is announced as winner.
7	A player completes a diagonal	That player is announced as winner.
8	All nine cells are filled without a win	Game Draw message is displayed.

19. ADVANTAGES, LIMITATIONS AND FUTURE ENHANCEMENTS

Advantages

- The program is easy to understand and suitable for beginners.
- It uses standard C++ input and output, so no extra library is required.
- The board is represented clearly with a two-dimensional array.
- Functions separate display and winning logic from the main game loop.
- Input validation prevents out-of-range positions and overwriting occupied cells.

Limitations

- The game supports only two players on the same computer.
- It uses a text-based interface rather than graphics.
- The game does not store scores or player history.
- There is no option to restart automatically after a game ends.
- The program expects numeric input; non-numeric input is not handled in this basic version.

Future Enhancements

- Add a menu to start, restart, or exit the game.
- Add player names and a scoreboard.
- Create a single-player mode with a computer opponent.
- Use classes and objects to demonstrate OOP more fully.
- Build a graphical interface with buttons and colours.
- Save match results to a file.

20. CONCLUSION AND BIBLIOGRAPHY

Conclusion

The Tic Tac Toe project successfully demonstrates how a familiar game can be implemented using basic C++ programming. The program stores the board in a two-dimensional array, displays it through a separate function, accepts moves from two players, validates the selected positions, and checks for winning combinations.

The project helped connect theoretical programming concepts with a practical application. It demonstrates the use of variables, arrays, functions, loops, conditional statements, Boolean return values, and console input/output. It also highlights the importance of planning program logic and testing different situations.

The current version is intentionally simple and focuses on correctness and readability rather than advanced features. It can be extended with a computer opponent, score tracking, a restart option, classes, and a graphical interface. Overall, it is a suitable beginner-level C++ mini project.

Bibliography / References

- Stroustrup, Bjarne. The C++ Programming Language. Addison-Wesley.
- cplusplus.com – C++ language and standard library reference: https://cplusplus.com/
- cppreference.com – C++ language and library reference: https://en.cppreference.com/
- Class notes and introductory C++ learning material.

Project prepared by: Sagar Gupta

UID: 25BCD10169

Chandigarh University